pfi

# Python for Fixed Income

COURSE 5C · RISK MODELS

# Monte Carlo Foundations

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# Course 5C: simulation, default, the unusual, and drift

### Week 1: Simulation and default

Monte Carlo, VaR, defaults, survival, hazards

### Week 2: Unusual data and drift

Outliers, isolation forest, PSI, monitoring

Today's case has an exact answer, so every simulated number can be checked.

**Today's question**

## How do you draw a loss you cannot write a formula for, and how far off can the answer be?

Simulate the uncertain parts many times, compute the loss each time, and read the distribution off the results.

**Draw, average, and say how far the average can be from the truth.**

# Names for a simulated number

Today's inputs are chosen for the lesson: a 2 percent default chance a year and a 60 percent loss given default.

### Allowed names

a simulated loss in US$

a Monte Carlo mean, with its error

an input chosen for the lesson

### Never called

a forecast of a loss

a real issuer's default chance

a view on any bond

**The bond is invented and its inputs are the course's. From day 3, default rates come from a synthetic cohort.**

# How Monte Carlo works

| | |
|---|---|
| **The problem** | A loss that depends on uncertain things has an average and a spread nobody wrote a formula for |
| **Draw** | Write down what is uncertain and how likely each outcome is, then draw it many times |
| **Average** | Compute the loss for each draw and average: many independent draws land close to the true mean |
| **You control** | The inputs, the number of trials, and the seed that fixes the draws |
| **What comes out** | Simulated losses: their mean, their spread, and their tail, with a measured error |

# One bond's loss in a year

Lorvebran Paper (99076YAC0), the book's largest BB holding: invented issuer, invented CUSIP.

**US$5,287,462**

exposure at default (market value)

**US$63,449.55**

expected loss: 0.02 x 0.60 x exposure

**US$444,146.85**

standard deviation of one year's loss

```
losses = np.where(rng.random(100_000) < PD, LGD * EAD, 0.0)
```

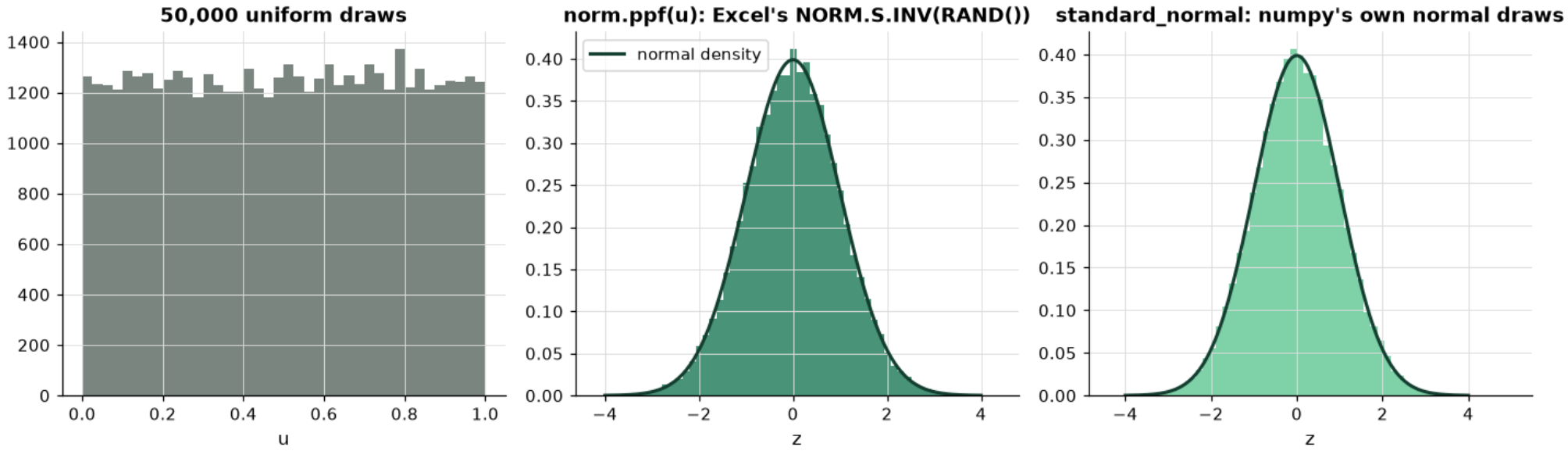
100,000 simulated years: 1,991 defaults, average loss US$63,164.03. In any one year the loss is zero or the full loss given default.

**The expected loss is an average over many years. That is what a simulation counts.**

# How a seed works

| | |
|---|---|
| **The problem** | A simulation has to give the same numbers every run, or nobody can check it |
| **The generator** | An algorithm turns a seed into a long stream that passes every test of randomness |
| **Same seed** | Two generators from one seed give the same draws in the same order: default_rng(SEED) |
| **Not the old way** | np.random.seed sets one hidden generator shared by all code; the course refuses it |
| **What comes out** | Every simulated loss reproduced on every run; spawn makes independent streams |

# From a uniform draw to a normal draw



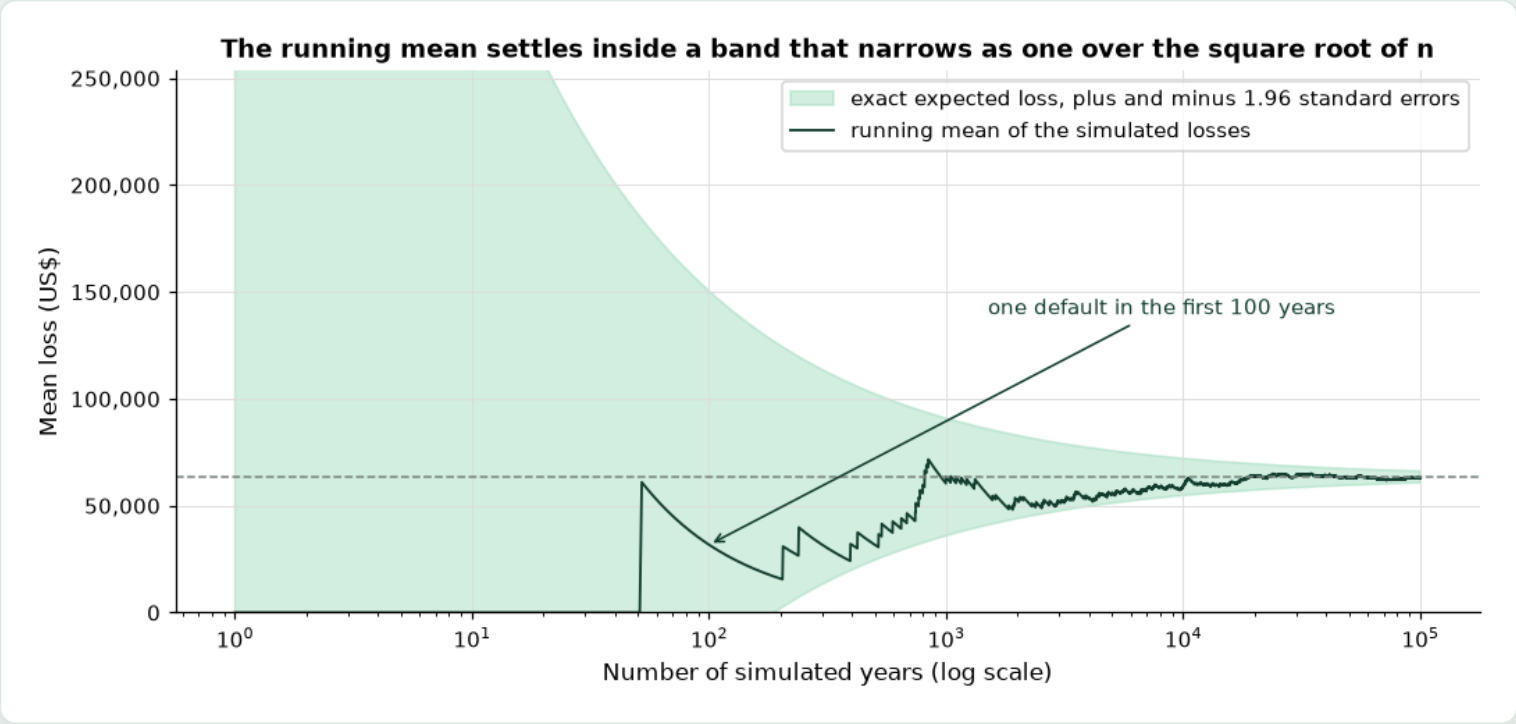

```
norm.ppf(u)    is Excel's    =NORM.S.INV(RAND())
```

**Same distribution, different draws. Neither is the normal sample.**

# How the standard error works

| | |
|---|---|
| **The problem** | A simulated mean is close to the truth; you need to say how close |
| **The law** | The mean of n independent draws lands, on average, on the true mean |
| **The error** | Its typical distance from the truth: the standard deviation of one draw over the square root of n |
| **You control** | n: four times the trials halve the error; a hundred times cut it by ten |
| **What comes out** | An estimate with a band: the mean plus and minus 1.96 standard errors (mc_mean) |

# The mean settles, at a price

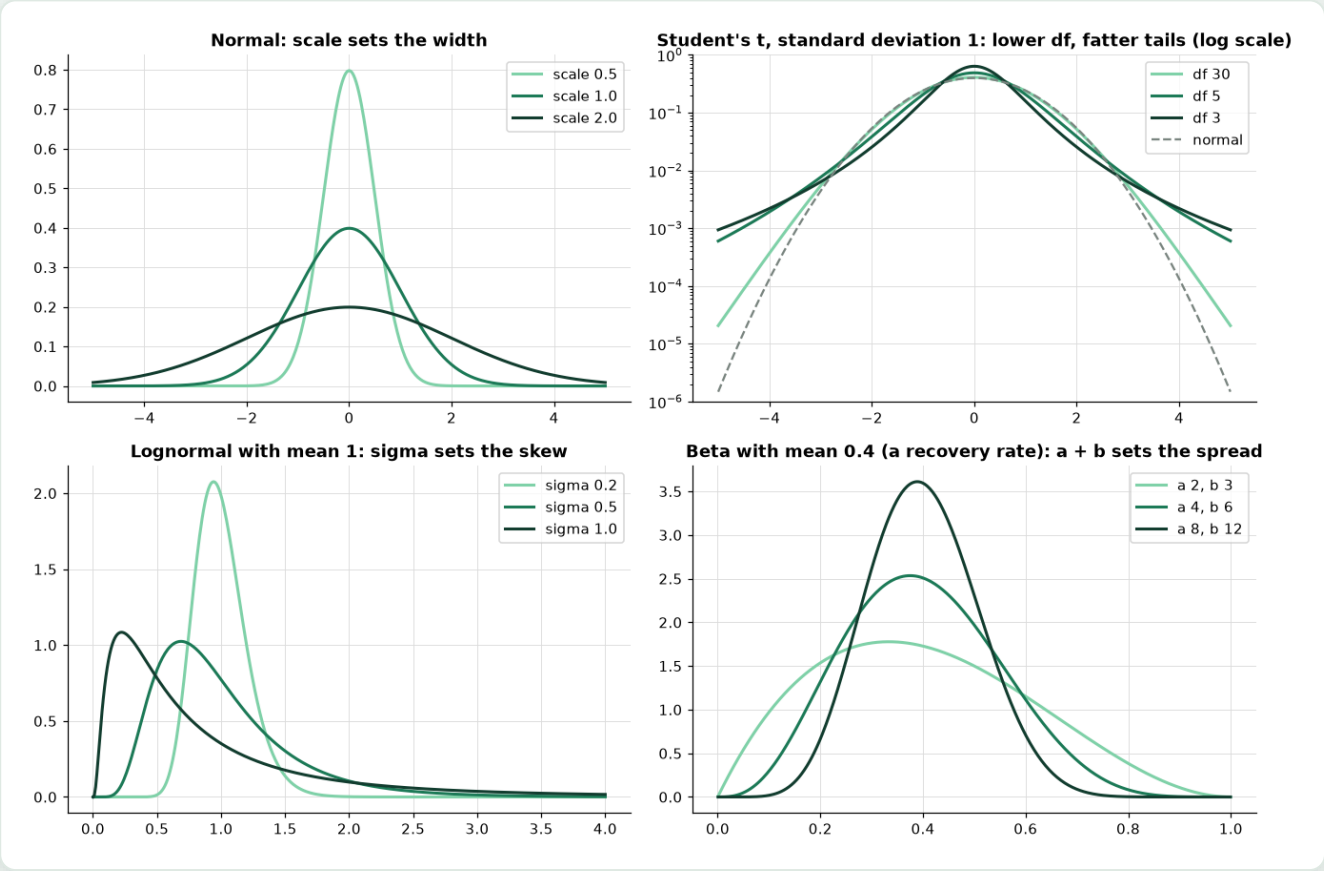



| Trials | Mean | Std error |
|---|---|---|
| 100 | 31,725 | 31,725 |
| 1,000 | 63,450 | 14,052 |
| 10,000 | 59,325 | 4,298 |
| 100,000 | 63,164 | 1,401 |

US$. Exact expected loss: US$63,449.55. 6 tests pass today.

**A hundred times the trials for ten times the precision: the square root is the price of Monte Carlo.**

# Distributions for risk inputs

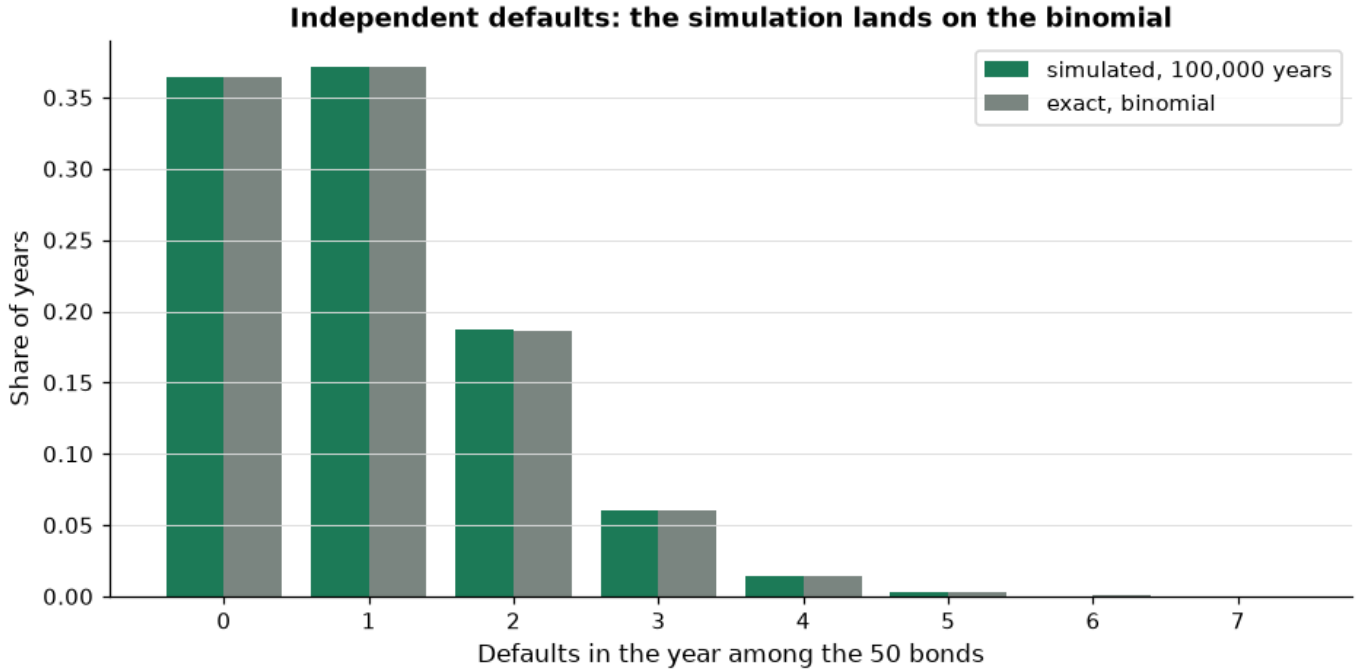



| Draws | Beyond 3 sd | x normal |
|---|---|---|
| normal | 0.00276 | 1.0 |
| t, df 30 | 0.00405 | 1.5 |
| t, df 5 | 0.01168 | 4.2 |
| t, df 3 | 0.01366 | 4.9 |

A million draws each, every distribution scaled to a standard deviation of 1.

**Same standard deviation, very different tail. Choose a distribution by its tail.**

# Counting defaults among 50 bonds




**0.3642**

years with no default (exact)

**0.0178**

4 or more defaults (exact); 0.0167 simulated

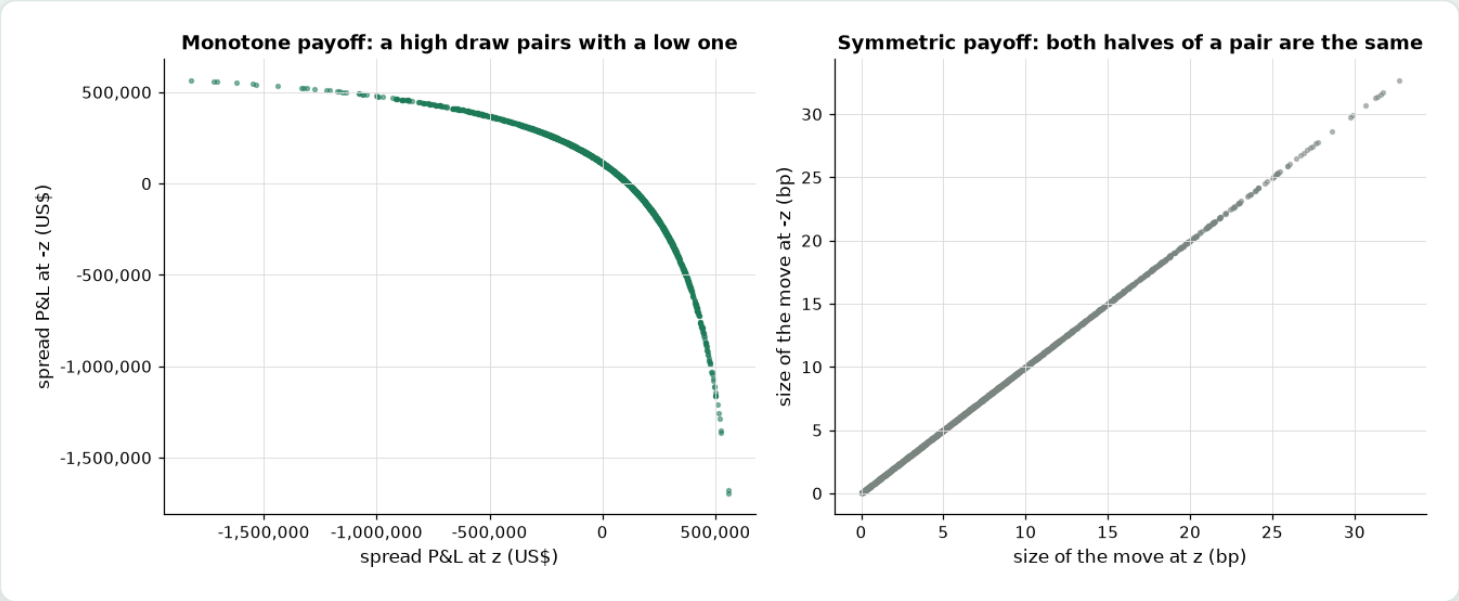
Each of 50 bonds defaults with a 2 percent chance, independently: the count is binomial, so the simulation can be checked.

**Real defaults cluster. Day 3 drops independence and the right tail grows.**

# How antithetic pairs work

| | |
|---|---|
| **The problem** | The standard error falls only as one over root n, so each draw has to count |
| **Mirror** | Draw half the normals, z, and use their mirror images, -z, for the other half |
| **Cancel** | When the payoff rises steadily with z, each pair's errors partly cancel |
| **You control** | Which payoff: it helps a monotone one and harms a symmetric one |
| **What comes out** | The same mean, a smaller error; average each pair before the standard error |

# Antithetic pairs: when they help




| 20,000 draws | Plain | Pairs | Ratio |
|---|---|---|---|
| Spread P&L (US$) | 2,218 | 821 | 7.31 |
| Move size (bp) | 0.04 | 0.06 | 0.52 |

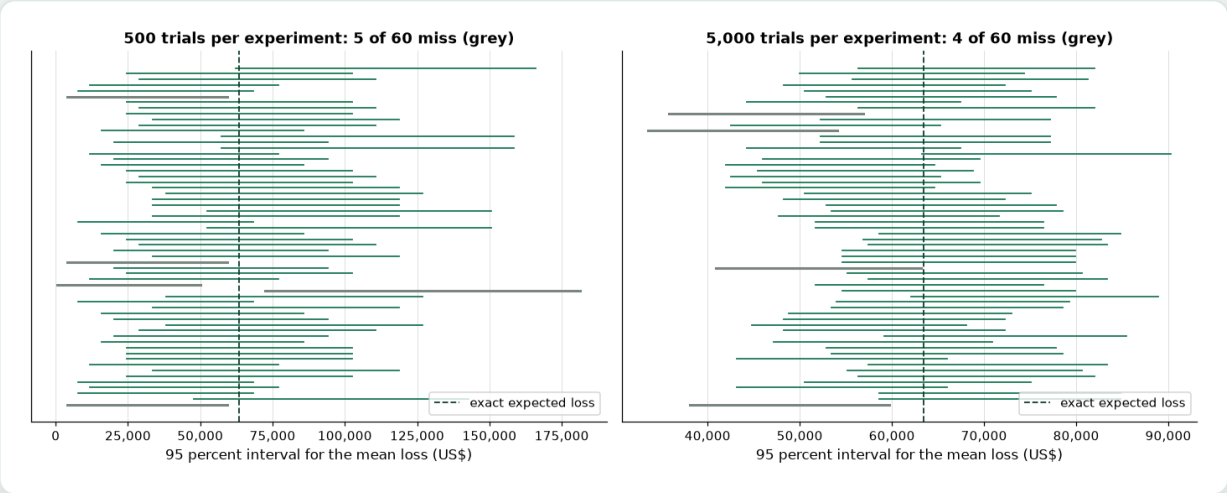
Standard errors, plain draws against antithetic pairs, and the variance ratio: above 1, the pairs win.

**Before using antithetic draws, ask whether the payoff moves one way as the draw rises.**

# How many trials?

```
n = ceil((1.96 * sd / h) ** 2)
```

| Half-width | 90% | 95% | 99% |
|---|---|---|---|
| 20% of EL | 3,315 | 4,706 | 8,128 |
| 10% of EL | 13,258 | 18,824 | 32,511 |
| 5% of EL | 53,029 | 75,293 | 130,044 |




**0.927**

held the truth: 500 trials

**0.955**

5,000 trials

**At 500 trials only 10 defaults are expected: the 1.96 rule overstates its own precision.**

# Where the name came from

**N. Metropolis and S. Ulam, September 1949**

## "The Monte Carlo Method"

Journal of the American Statistical Association, volume 44, issue 247, pages 335 to 341

Metropolis wrote that he chose the name, "not unrelated to the fact that Stan had an uncle who would borrow money from relatives because he 'just had to go to Monte Carlo.'" Von Neumann outlined the method in a letter of March 11, 1947.

Source: N. Metropolis, "The Beginning of the Monte Carlo Method", Los Alamos Science, Special Issue 1987, pages 125 to 130 (mcnpx.lanl.gov). Read 22 October 2026.

# Excel side by side

**Excel side by side (tested in Excel)**

| | |
|---|---|
| `=E1*E2*E3` | expected loss US$63,450 |
| `=RAND()` | uniform, new on every recalculation |
| `=NORM.S.INV(A2)` | normal draw: norm.ppf within 1e-9 |
| `=NORM.S.INV(1-A2)` | the antithetic draw |
| `=IF(A2<$E$1,$E$2*$E$3,0)` | one year's loss, filled down |
| `=STDEV.S(C2:C1001)/SQRT(COUNT(C2:C1001))` | standard error US$14,052 |
| `=1-BINOM.DIST(3,50,E1,TRUE)` | at least 4 of 50: 0.017758 |
| `=ROUNDUP((NORM.S.INV(0.975)*E5/(0.1*E4))^2,0)` | 18,824 trials |

**A one-variable Data Table repeats the sheet once per row: 1,000 rows, a fresh draw in each. Excel has no seed.**

# Save the day: the new module

### 1. Init

The work folder, local settings, the guard

### 2. .gitignore

__pycache__/, .pytest_cache/, *.xlsx

### 3. Commit

mlkit.py, riskkit.py, and its tests

```
Day 1: riskkit with mc_mean, trials_needed, and antithetic_normals
```

**The book and plotkit stay untracked: they are the course's data and helpers, not your code.**

# The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

## Docstring

What is drawn, from which inputs, in which unit: what you asked for.

→

2

## Code back

Written for the course in the style of an AI answer. Read it.

→

3

## Test first

Assert what the docstring promises. Run it: it fails.

→

4

## Fix

Find the rule it broke, fix it, run the test again.

**Today: a generator seeded inside the loop: 0 defaults in 1000 years, a standard error of zero.**

*Code that runs is not the same as code that is right.*

# What you can do now

- Simulate a loss and check it against PD x LGD x EAD
- Seed a generator, reproduce every draw, spawn streams
- Report a mean with its standard error (mc_mean)
- Choose a distribution by its tail
- Use antithetic pairs where they help; size a run (trials_needed)

**Week 1, Day 2** Historical simulation and VaR